University of Bologna
Master's Degree in Computer Science and Engineering

Lab-activity-04

Laboratory Report

Student:
Sajmir Buzi

Academic Year 2025/2026

0.1 Objective

The objective of this activity is to implement a behaviour-based controller in ARGoS using the motor schemas architecture. The robot has to reach a light source while avoiding collisions with walls, boxes and other robots.
The controller is written in Lua and uses the foot-bot light sensors, proximity sensors, LEDs and differential steering wheels. The final behaviour is obtained by combining simple motor schemas represented as vectors in polar coordinates.

0.2 Motor Schemas

The controller is based on three main schemas: cruise, light attraction and obstacle repulsion. Each schema returns a vector with two fields: `length` and `angle`. The vectors are then summed using the function `vector.vec2_polar_sum`.
The cruise schema gives the robot a small forward force, with length `0.5` and angle `0.0`. This prevents the robot from stopping when the light stimulus is weak.
The light attraction schema reads all light sensors. For every active sensor, it creates a vector with the same direction as the sensor and with a length proportional to the perceived light value. All these vectors are summed to obtain the final attraction vector.
The obstacle avoidance schema reads the proximity sensors and creates repulsive vectors in the opposite direction of the detected obstacles. The proximity value is squared and multiplied by `2.0`, so close obstacles have a stronger effect than distant ones.

0.3 Target Behaviour

When the robot gets very close to the light, simply following the light vector may cause oscillations. This happens because the perceived direction of the light changes quickly when the robot is near the source.
To avoid this problem, the function `near_light()` checks the maximum value perceived by the light sensors. If this value is greater than `NEAR_LIGHT_VALUE`, the robot is considered to be near the target. When this condition is true and no obstacle is very close, the robot stops translating and rotates slowly on itself by setting opposite wheel velocities. This keeps the robot close to the light source without moving far away from the target area.

0.4 Wheel Velocity Computation

In each simulation step, the controller computes the cruise vector, the light attraction vector and the obstacle repulsion vector. The three vectors are summed to obtain the final desired motion direction.
The resulting vector is converted into a translational velocity `v` and an angular velocity `omega`. Then these values are converted into left and right wheel velocities using the differential steering equations.

The laboratory specification states that the wheel velocity cannot exceed `15`. For this reason, after computing the two wheel velocities, the controller scales them if one of them is above the maximum allowed value. This keeps the direction of the action while respecting the physical constraint.

0.5 Conclusion

The motor schemas architecture has the advantage of being simple, modular and easy to extend. Each behaviour can be designed separately as a vector field, and the final action is obtained by combining the resulting vectors. This makes the controller intuitive and effective for reactive tasks such as light seeking and obstacle avoidance. However, this approach also has some limitations. Since the robot reacts only to the current sensor readings, it has no memory of past situations and cannot explicitly plan a path. For this reason, in some configurations the robot may get stuck near obstacles or enter cyclic behaviours, especially when attraction to the light and obstacle repulsion generate conflicting commands.
Compared with the subsumption architecture, motor schemas are easier to compose because behaviours are combined continuously through vector summation. Subsumption is often easier to understand at the level of priorities, because higher-level behaviours can suppress lower-level ones, but it may become less flexible when many behaviours must interact at the same time. The performance of both approaches can be assessed by running several simulations with different initial positions and measuring indicators such as the time needed to reach the light, the final distance from the target and the number of collisions.
So motor schemas are particularly suitable because new behaviours can be added as additional vectors with appropriate weights. A hybrid architecture could also be designed by combining the two ideas: motor schemas could generate smooth motion commands, while a subsumption layer could manage special cases or priorities, such as stopping near the target or escaping from a blocked situation. In my opinion it would be useful if the task required both continuous motion control and discrete decisions.